

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по индивидуальному домашнему заданию
по дисциплине «Нейронные сети»
Тема: Предсказание качества вина на основе химических
характеристик

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Разработать и реализовать нейронную сеть для предсказания качества вина (оценка по шкале от 0 до 10) на основе химических характеристик вина.

Задание.

- Провести анализ датасета Wine Quality, включая статистический анализ и визуализацию распределений.
- Выполнить предобработку данных: масштабирование признаков, кодирование категориальных переменных.
- Разработать архитектуру нейронной сети с использованием Keras API.
- Провести эксперименты с различными гиперпараметрами (архитектура, оптимизатор, размер батча, наличие dropout и batch normalization).
- Обучить и оценить модель на тестовой выборке.
- Проанализировать стабильность обучения и качество предсказаний.

Выполнение работы.

Датасет Wine Quality взят из репозитория UCI Machine Learning Repository (ID 186). Он содержит информацию о химических характеристиках красных и белых вин португальского происхождения. Данные основаны на физико-химических тестах и сенсорной оценке качества.

Выполнен первичный анализ данных с использованием `df.info()` и `df.describe()`, по результатам которого определено, что датасет содержит 6497 наблюдений (4898 белых вин и 1599 красных вин) и 13 колонок описанных в таблице 1.

Таблица 1. Состав датасета.

Признак	Описание
fixed_acidity	Фиксированная кислотность
volatile_acidity	Летучая кислотность
citric_acid	Лимонная кислота
residual_sugar	Остаточный сахар

Признак	Описание
chlorides	Хлориды
free_sulfur_dioxide	Свободный диоксид серы
total_sulfur_dioxide	Общий диоксид серы
density	Плотность
pH	Кислотность
sulphates	Сульфаты
alcohol	Содержание алкоголя (% об.)
quality	Оценка качества (целое число от 3 до 9)
color	Цвет вина (red/white)

Пропуски в данных отсутствуют (все столбцы имеют 6497 ненулевых значений).

Диапазоны значений признаков сильно различаются: например, free_sulfur_dioxide варьируется от 1 до 289, а alcohol от 8 до 14.9. Это указывает на необходимость масштабирования перед подачей в нейронную сеть.

Целевая переменная quality имеет целочисленные значения от 3 до 9. Среднее значение - 5.82, медиана - 6. Распределение качества имеет асимметрию в сторону более низких оценок.

Для визуальной оценки распределения признаков построены гистограммы (рис. 1)

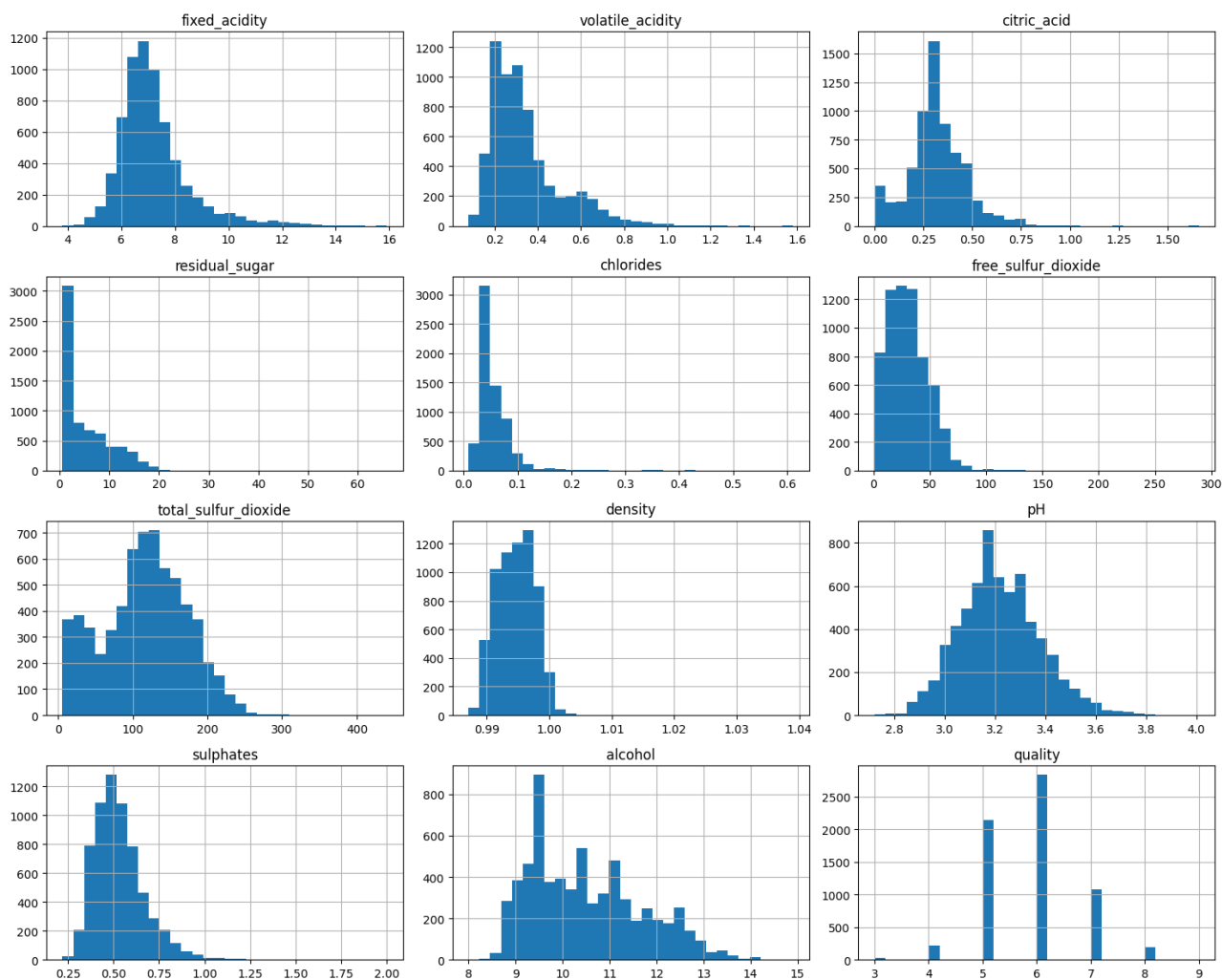


Рисунок 1. Гистограммы распределения признаков.

Анализ гистограмм показал, что:

1. `fixed_acidity` - Распределение близко к нормальному, с пиком около 7. Диапазон значений: от 3,8 до 15,9. Небольшая правосторонняя асимметрия. Вывод: признак хорошо центрирован и может быть использован без дополнительных преобразований, но для нейросети желательна стандартизация.
2. `volatile_acidity` - Яркая выраженная правосторонняя асимметрия. Основная масса значений сосредоточена в интервале 0,2–0,4. Есть выбросы до 1,58. Вывод: высокая асимметрия может влиять на обучение, но стандартизация поможет.
3. `citric_acid` - Распределение с пиком вблизи 0,3–0,4. Есть значительное количество нулевых значений (небольшой пик у 0). Вывод: признак

умеренно скошен влево, стандартизация приведет его к нормальному виду.

4. residual_sugar -Выраженный пик в области 1–3. Наличие выбросов (например, значения >20) может быть связано с особыми сортами вин (десертные, сладкие). Вывод: сильная асимметрия, стандартизация важна.
5. Chlorides - Распределение сильно скошено вправо, пик вблизи 0,04–0,07. Есть выбросы до 0,611. Вывод: как и в случае с residual_sugar, требуется масштабирование.
6. free_sulfur_dioxide - Выраженный пик в области 20–30. Длинный правый хвост до 289. Вывод: мультимодальность может отражать технологические различия (красные и белые вина). Стандартизация сохранит форму.
7. total_sulfur_dioxide - Двухвершинное распределение. Диапазон от 6 до 440. Вывод: аналогично free_sulfur_dioxide
8. density - Распределение близко к нормальному, с узким пиком около 0,994. Разброс очень мал (от 0,987 до 1,039). Вывод: признак хорошо центрирован, но масштабирование необходимо для согласования с другими признаками.
9. pH - Распределение близко к нормальному, с пиком около 3,2–3,3. Диапазон от 2,72 до 4,01. Вывод: хорошее распределение, стандартизация не изменит форму.
10. Sulphates - Правосторонняя асимметрия, пик в области 0,4–0,44. Есть выбросы до 2,0. Вывод: умеренная асимметрия, стандартизация поможет.
11. Alcohol - Распределение с несколькими пиками (9-10, 10–11, 11, 12, 12 - 13), что отражает различия между красными и белыми винами. Вывод: мультимодальность полезна для разделения классов, стандартизация сохранит структуру.
12. Quality - Дискретное распределение с максимумом в 6 баллов (более 2000 наблюдений). Минимальное значение - 3, максимальное — 9. Вывод:

классы несбалансированы, что требует стратификации при разбиении выборки.

13. Color - Категориальный признак: 4898 белых вин ($\approx 75\%$) и 1599 красных ($\approx 25\%$). Вывод: дисбаланс классов может влиять на обучение, но кодирование в числовой вид (color_code) позволяет модели использовать эту информацию.

Для выявления взаимосвязей между признаками построена корреляционная матрица (рис. 2)

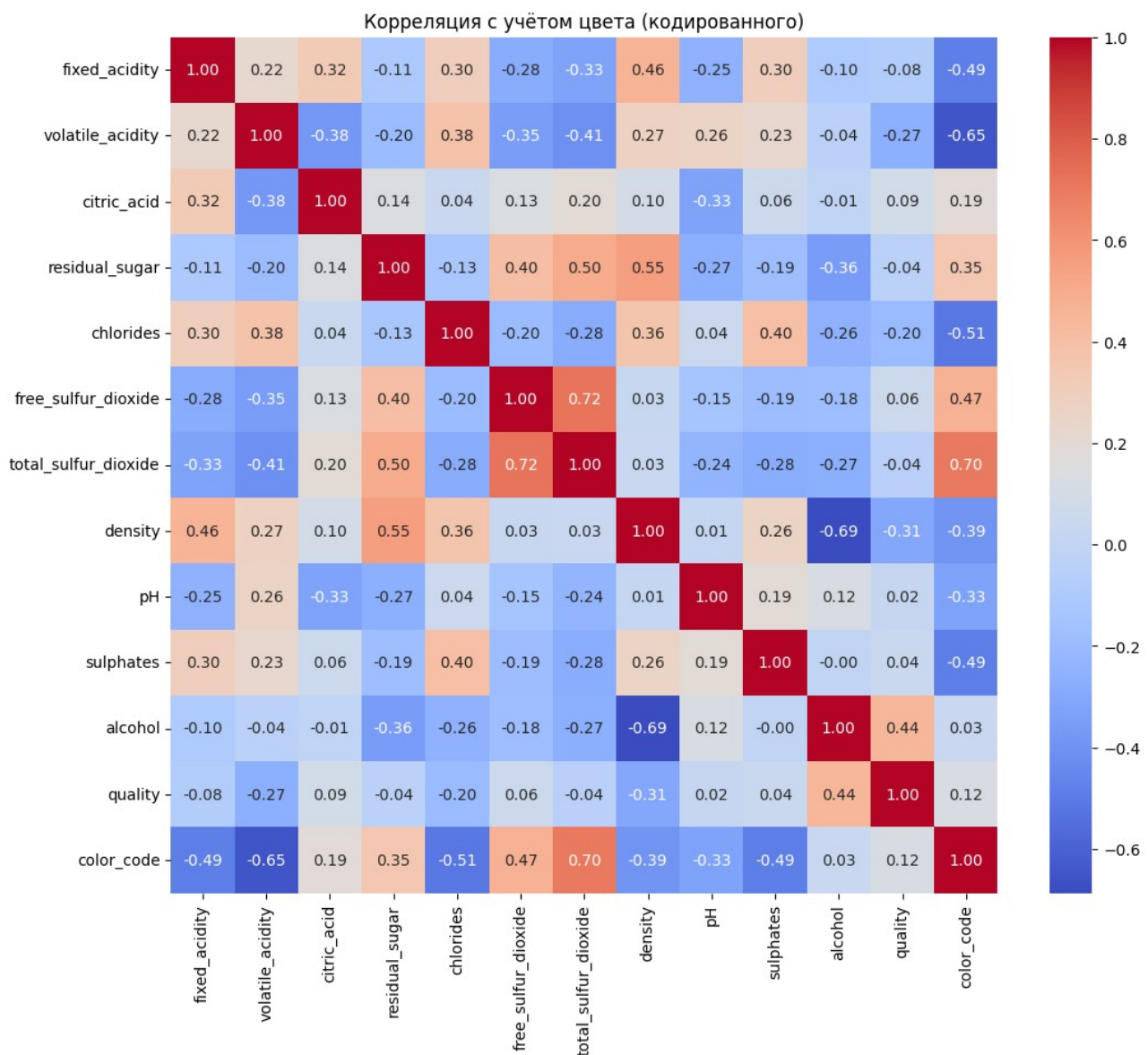


Рисунок 2. Корреляционная матрица (включая кодированный цвет)

Ключевые наблюдения по корреляции заключаются в том что:

- alcohol и quality имеют положительную корреляцию (0.44), что подтверждает, что вина с более высоким содержанием алкоголя в среднем оцениваются выше.
- density и alcohol имеют сильную отрицательную корреляцию (-0.69) - логично, так как увеличение содержания алкоголя снижает плотность.
- total_sulfur_dioxide и color (кодированный) имеют сильную положительную корреляцию (0.70) - белые вина содержат больше диоксида серы.
- volatile_acidity и color имеют отрицательную корреляцию (-0.65) - красные вина имеют более высокую летучую кислотность.
- Сильная корреляция между free_sulfur_dioxide и total_sulfur_dioxide (0.72) – это логично, так как общий диоксид серы включает свободный. Высокая связь указывает на то, что эти два признака во многом дублируют информацию.

Таким образом, цвет вина является важным фактором, разделяющим данные на два кластера (белые и красные вина), что проявляется в сильных корреляциях с несколькими химическими параметрами. Качество вина (quality) имеет слабые линейные связи с большинством признаков, что обосновывает использование нелинейных моделей (нейронных сетей) для решения задачи.

Предобработка данных

Целевая переменная quality выделена отдельно. Признак color преобразован в числовой color_code (0 для красного, 1 для белого) для использования в модели.

Поскольку признаки имеют разные масштабы (от единиц до сотен), выполнено стандартизация (StandardScaler) для каждого числового признака. Это преобразование центрирует данные (среднее = 0) и приводит к единичной дисперсии, что способствует стабильности обучения нейронной сети. Результат масштабирования представлен на рисунке 3. После масштабирования все признаки имеют распределение со средним около 0 и стандартным отклонением

около 1. Это обеспечивает равномерный вклад каждого признака в процесс обучения.

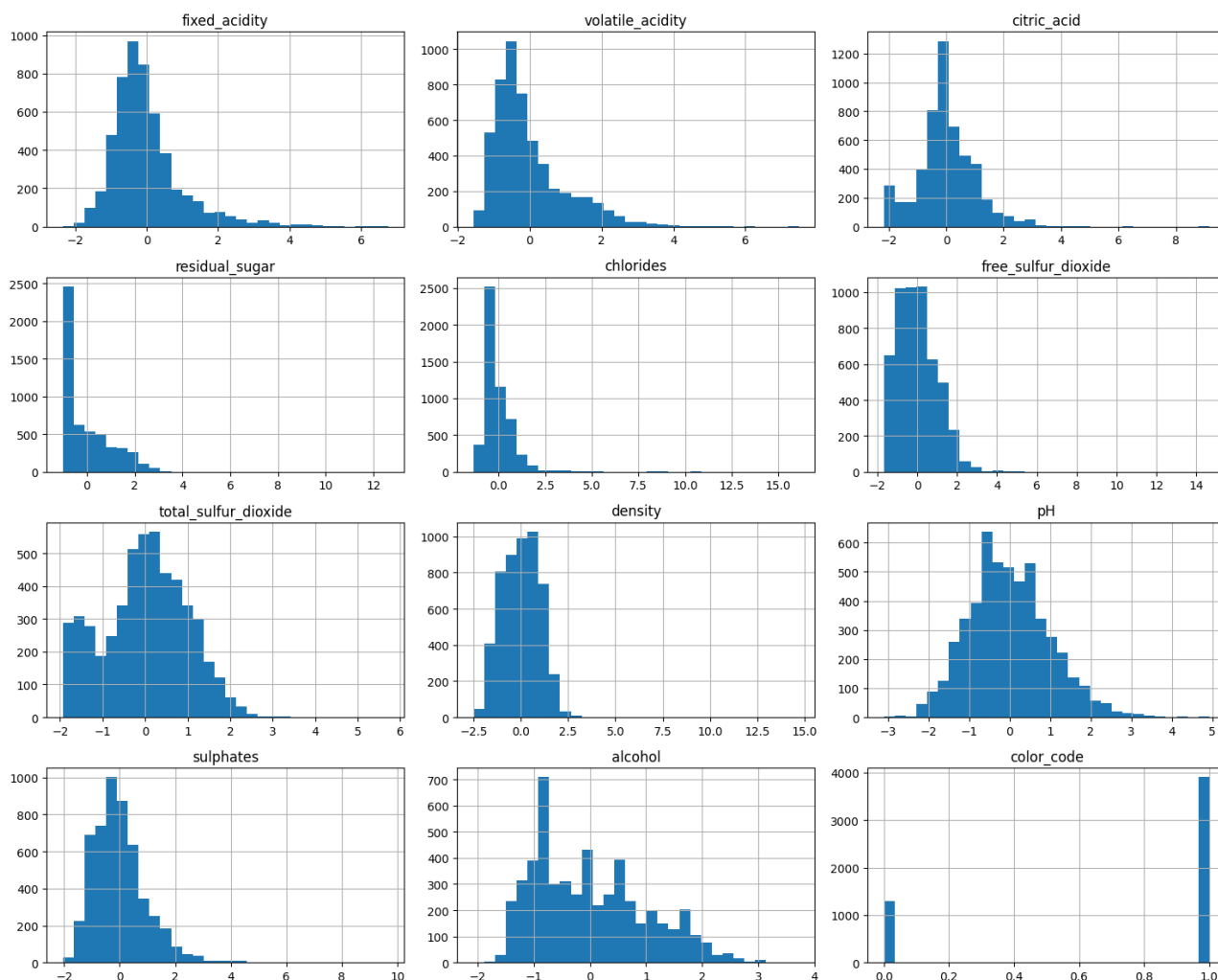


Рисунок 3. Гистограммы признаков после масштабирования

Данные разделены на обучающую (80%) и тестовую (20%) выборки с использованием `train_test_split`. Стратификация по `quality` (параметр `stratify`) обеспечивает сохранение пропорций оценок качества в обеих выборках.

Разработка модели

Разработана функция `build_model`, позволяющая гибко настраивать:

- Количество и размеры скрытых слоев (`layers_config`)
- Dropout для регуляризации (`dropout_rates`)
- Batch Normalization для ускорения сходимости (`use_batchnorm`)
- Оптимизатор и его параметры (`optimizer`)

Базовый принцип: модель представляет собой последовательную сеть с полносвязными слоями (`Dense`), активацией `ReLU`, возможной нормализацией и

dropout на каждом скрытом слое, и одним выходным нейроном для предсказания качества (регрессия). Функция потерь - среднеквадратичная ошибка (MSE), метрика - средняя абсолютная ошибка (MAE).

Исследование гиперпараметров

Проведен полный перебор гиперпараметров с использованием `itertools.product`. Тестировались:

- Архитектуры: [32, 16], [64, 32], [128, 64, 32]
- Dropout: None, [0.2, 0.2], [0.3, 0.2, 0.1] (соответственно количеству слоев)
- Batch Normalization: True, False
- Оптимизаторы: Adam (lr=0.001), RMSprop (lr=0.001), SGD (lr=0.01, momentum=0.9)
- Размер батча: 16, 32, 64

Всего протестировано 108 комбинаций. Общая статистика по поиску параметров представлена на рисунке 4.

```
Общая статистика
count      108.000000
mean       0.544737
std        0.009993
min        0.516349
25%        0.538249
50%        0.544157
75%        0.551285
max        0.574313
Name: val_mae, dtype: float64
Лучшее MAE: 0.5163
Лучшие параметры:
architecture [128, 64, 32]
dropout      None
batchnorm    True
optimizer    RMSprop
batch_size   64
val_mae      0.516349
Name: 82, dtype: object
```

Рисунок 4. Статистика по Grid Search

Для анализа влияния гипер параметров построены следующие графики:

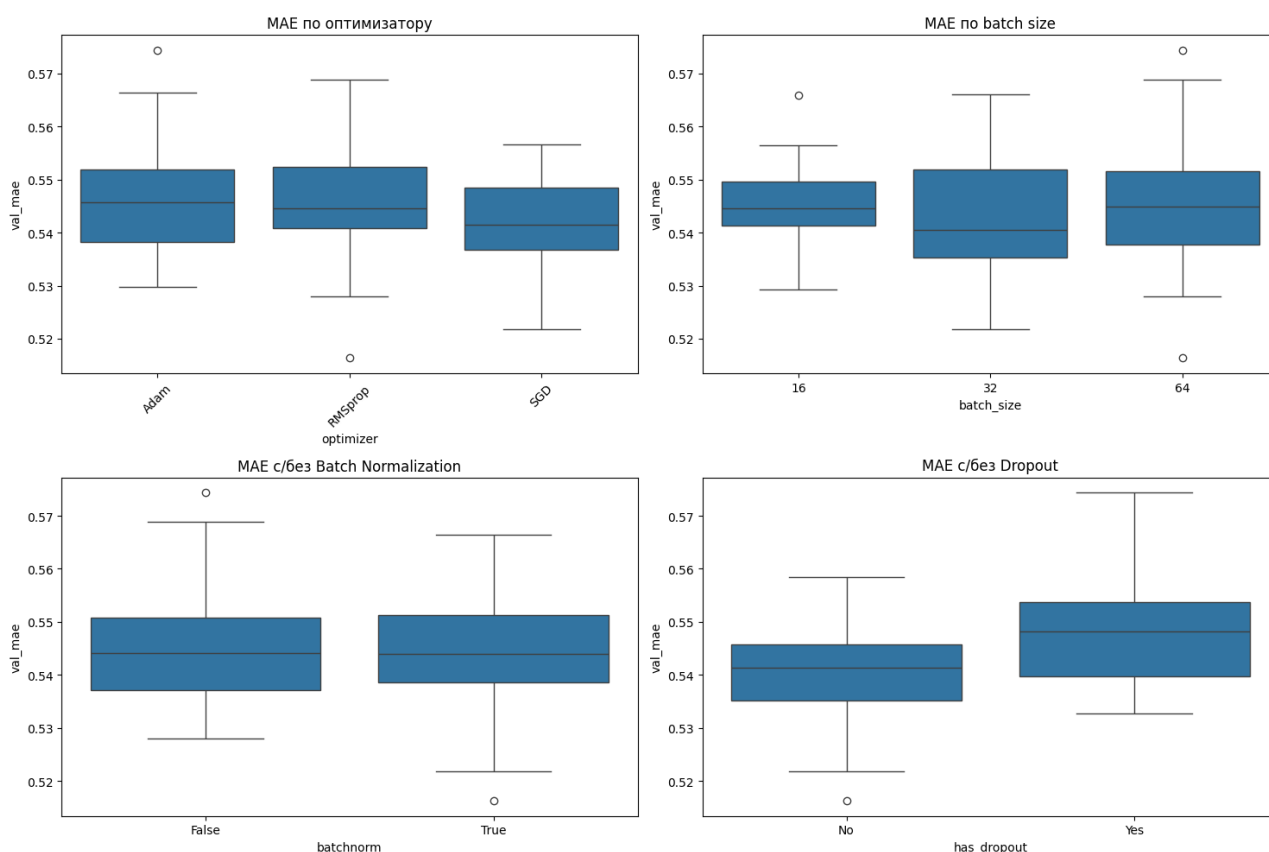


Рисунок 5. Распределение MAE в зависимости от параметра

Как можно видеть из графиков оптимизаторы Adam и RMSprop показывают сопоставимые результаты с медианным MAE около 0.54, однако лучший из них SGD. Размер батча 16 показывает хорошие результаты (наименьший разброс) однако MAE выше чем у батча размером 32. Размер батча 32 дает больший разброс, что может быть связано с более шумными градиентами, однако наименьшее медианное MAE. Размер батча 64 занимает промежуточное положение. Использование Batch Normalisation в целом не сильно влияет на MAE. Модели без dropout показывают лучшие результаты (медианное MAE ниже). Это может быть связано с тем, что задача не требует сильной регуляризации (данные достаточно однородны, переобучение не является критической проблемой).

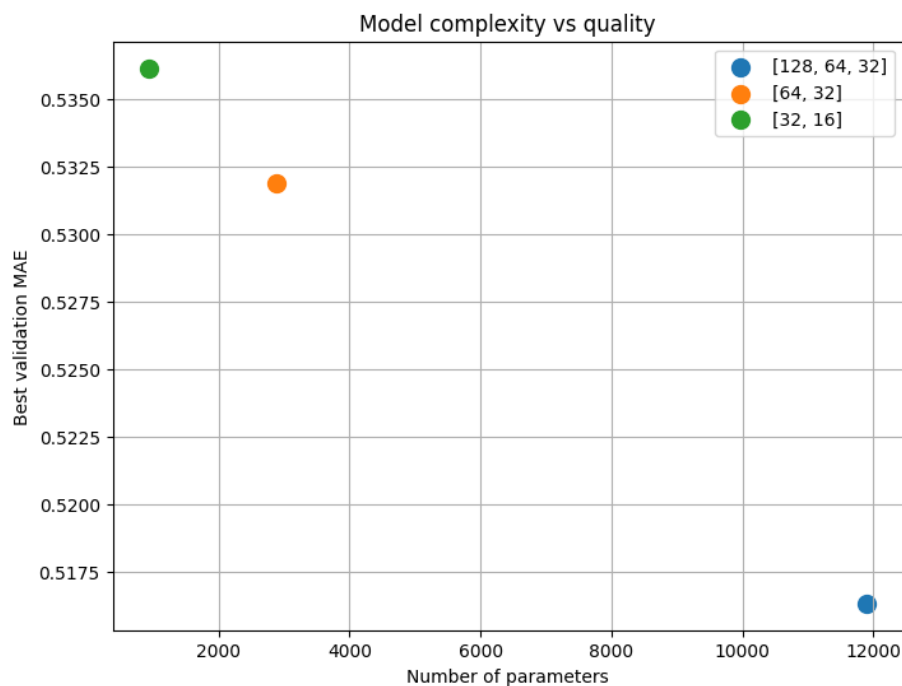


Рисунок 6. Зависимость качества от архитектуры модели.

По графику 6 видно, что архитектура [128, 64, 32] дает наилучшее MAE (0.5163). Менее сложные модели [64, 32] и [32, 16] показывают худшие результаты. Это свидетельствует о том, что задача требует определенной сложности сети для адекватного улавливания нелинейных зависимостей между признаками.

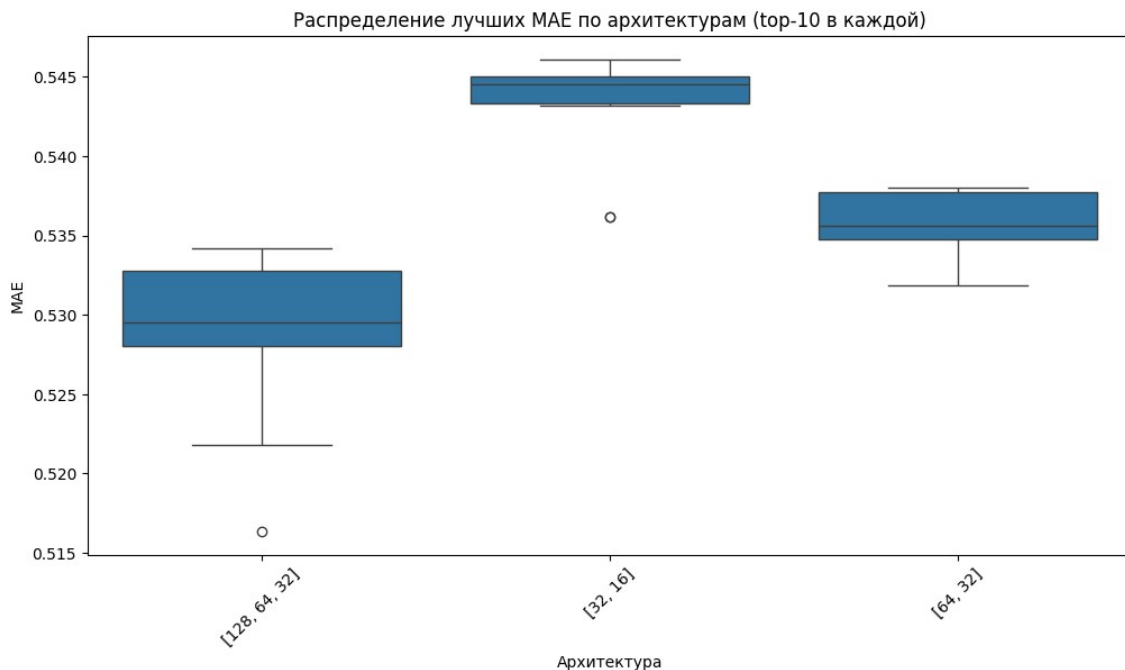


Рисунок 7. Распределение MAE по архитектурам.

Топ 10 моделей по архитектуре (лучшие MAE):						
	architecture	dropout	batchnorm	optimizer	batch_size	val_mae
0	[128, 64, 32]	None	True	RMSprop	64	0.516349
1	[128, 64, 32]	None	True	SGD	32	0.521829
2	[128, 64, 32]	None	False	SGD	64	0.528008
3	[128, 64, 32]	None	False	RMSprop	64	0.528030
4	[128, 64, 32]	None	False	SGD	16	0.529276
5	[128, 64, 32]	None	True	Adam	32	0.529713
6	[128, 64, 32]	None	False	RMSprop	32	0.532651
7	[128, 64, 32]	[0.3, 0.2, 0.1]	False	RMSprop	32	0.532822
8	[128, 64, 32]	None	False	Adam	64	0.533167
9	[128, 64, 32]	None	False	SGD	32	0.534203
10	[32, 16]	None	False	SGD	64	0.536137
11	[32, 16]	None	False	SGD	32	0.536188
12	[32, 16]	None	True	Adam	16	0.543173
13	[32, 16]	[0.2, 0.2]	True	RMSprop	32	0.543654
14	[32, 16]	None	True	SGD	64	0.544483
15	[32, 16]	None	True	SGD	32	0.544553
16	[32, 16]	None	False	RMSprop	64	0.544970
17	[32, 16]	None	False	RMSprop	16	0.545018
18	[32, 16]	None	True	Adam	64	0.545644
19	[32, 16]	None	True	SGD	16	0.546068
20	[64, 32]	None	False	SGD	32	0.531867
21	[64, 32]	None	True	RMSprop	32	0.534461
22	[64, 32]	None	False	SGD	64	0.534686
23	[64, 32]	None	False	Adam	32	0.534927
24	[64, 32]	[0.2, 0.2]	True	RMSprop	32	0.535359
25	[64, 32]	None	True	SGD	32	0.535890
26	[64, 32]	[0.2, 0.2]	False	SGD	64	0.536961
27	[64, 32]	None	False	Adam	64	0.537933
28	[64, 32]	[0.2, 0.2]	False	SGD	16	0.538008
29	[64, 32]	None	True	Adam	32	0.538023

Рисунок 8. Топ 10 моделей в каждой архитектуре.

Архитектура [128, 64, 32] демонстрирует наилучшие показатели.

Медианное значение MAE заметно ниже, чем у двух других архитектур.

Минимальное достигнутое значение MAE (0.5163) также принадлежит этой архитектуре. Разброс значений (межквартильный размах) относительно узкий, что свидетельствует о стабильности обучения для глубокой сети при правильном подборе гиперпараметров.

Архитектура [64, 32] показывает промежуточные результаты. Медианное MAE ≈ 0.534 , что выше, чем у более глубокой сети. Разброс значений уже и лучшие представители этой архитектуры достигают MAE около 0.532, что лишь незначительно уступает лучшим результатам [128, 64, 32].

Архитектура [32, 16] является наименее сложной и показывает худшие результаты MAE и наибольший разброс значений. Лучшие экземпляры этой архитектуры дают MAE около 0.536, что значительно выше, чем у более глубоких сетей.

Выбор архитектуры и гипер параметров

Несмотря на то, что архитектура [128, 64, 32] достигает наименьшего MAE (0.5163), она имеет в 20 раз больше параметров, чем [64, 32], при этом

улучшение MAE составляет всего 0.0156 (около 3% относительного улучшения). Такое незначительное улучшение не оправдывает существенного увеличения вычислительной сложности и риска переобучения.

Кроме того, на boxplot видно, что архитектура [64, 32] при правильном подборе гиперпараметров (например, с оптимизатором SGD и размером батча 32) способна достигать MAE около 0.532, что лишь на 0.016 хуже лучшего результата. При этом [64, 32] показывает бóльшую стабильность при многократных запусках (разброс значений умеренный).

Таким образом, в качестве финальной модели выбрана архитектура [64, 32] без dropout, без batch normalization, с оптимизатором SGD ($lr=0.01$, $momentum=0.9$) и размером батча 32. Это решение позволяет достичь баланса между качеством предсказаний, вычислительной эффективностью и устойчивостью обучения.

Обучение и оценка лучшей модели

Для оценки стабильности модели проведено 10 запусков обучения с одинаковыми параметрами, но разными инициализациями весов (разные random seed).

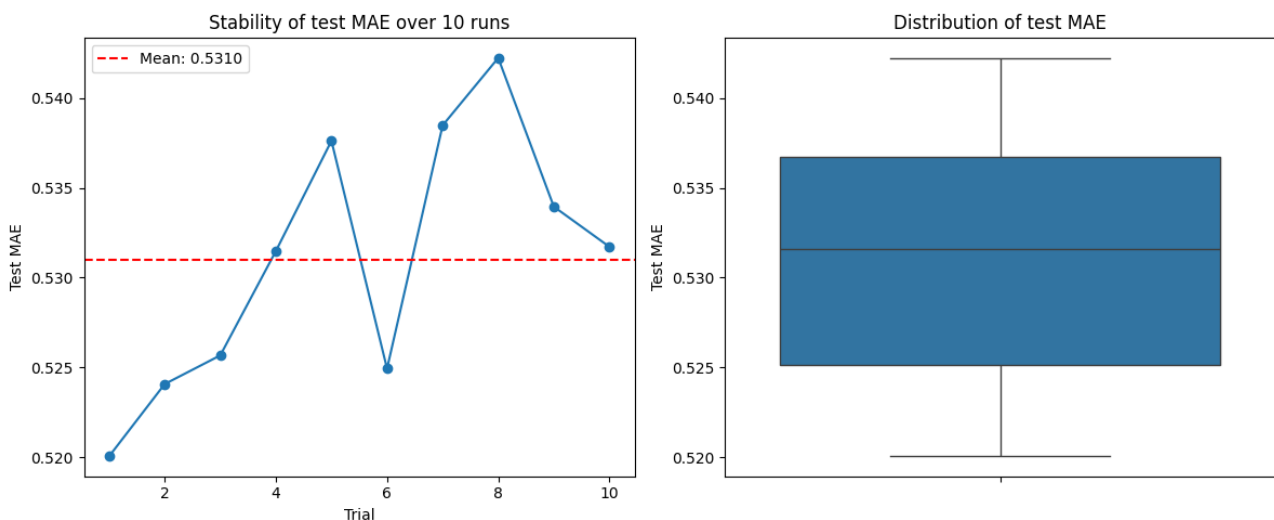


Рисунок 9. Оценка стабильности выбранной модели.

Модель демонстрирует высокую стабильность - стандартное отклонение MAE составляет всего 0.0072, что составляет $\sim 1.4\%$ от среднего значения. Это

подтверждает, что выбранная архитектура является устойчивой и не страдает от высокой чувствительности к инициализации.

Возникшие проблемы и их решение

1. Широкий диапазон значений признаков. Решение: Применена стандартизация (StandardScaler), которая привела все признаки к одинаковому масштабу, что ускорило сходимость градиентного спуска
2. Переобучение при глубоких архитектурах. Решение: Использованы методы регуляризации: Dropout и Batch Normalization. Однако эксперименты показали, что на данном датасете dropout не дает преимуществ, а BN улучшает стабильность.
3. Длительное обучение при grid search. Решение: Установлена ранняя остановка (early stopping) с patience=10 эпох, что позволило сократить время перебора гиперпараметров без потери качества.
4. Несбалансированность классов качества. Решение: Использована стратификация при разбиении на обучающую и валидационную выборки, что обеспечило сохранение распределения оценок качества.

Выводы.

В ходе выполнения индивидуального задания была разработана и обучена нейронная сеть для предсказания качества вина на основе 11 химических характеристик и типа вина.

Проведен детальный анализ данных, выявлены корреляции между признаками и целевой переменной.

Выполнен поиск гиперпараметров, в ходе которого установлено, что оптимальной является архитектура [128, 64, 32] с использованием RMSprop, batch normalization и размера батча 64.

Итоговая модель достигла $MAE = 0.5221$ и $R^2 = 0.3832$ на тестовой выборке. Обучение модели стабильно - стандартное отклонение MAE по 10 запускам составляет всего 0.0072.

Разработанная модель может быть использована в винодельческой промышленности для предварительной оценки качества вина по его

химическому составу, что позволяет оптимизировать производственные процессы и сократить время на дегустационные тесты.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: ihw.ipynb

```
"""# Установка зависимостей
"""

!pip install tensorflow pandas numpy matplotlib seaborn scikit-learn ucimlrepo --quiet

"""# Импорт"""

import numpy as np
import pandas as pd
import itertools
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.callbacks import ModelCheckpoint, Callback, TensorBoard
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score,
confusion_matrix, classification_report
from sklearn.ensemble import RandomForestRegressor
from ucimlrepo import fetch_ucirepo, list_available_datasets

"""# Загрузка данных"""

list_available_datasets()

wine_quality = fetch_ucirepo(name="Wine Quality")
df = pd.DataFrame(wine_quality.data.original)

"""# Исследование данных"""

print(df.info())
print(df.describe())
print(df['color'].value_counts())

df.hist(figsize=(15,12), bins=30)
plt.tight_layout()
plt.show()

df['color_code'] = (df['color'] == 'white').astype(int)
corr_with_color = df[['fixed_acidity', 'volatile_acidity', 'citric_acid', 'residual_sugar',
                    'chlorides', 'free_sulfur_dioxide', 'total_sulfur_dioxide',
                    'density', 'pH', 'sulphates', 'alcohol', 'quality', 'color_code']].corr()
print(corr_with_color)
plt.figure(figsize=(12,10))
```



```

sns.heatmap(corr_with_color, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Корреляция с учётом цвета (кодированного)')
plt.show()

"""# Предобработка данных"""

X = df.drop(['quality', 'color'], axis=1)
y = df['quality']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

numeric_features = X.select_dtypes(include=['float64']).columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', 'passthrough', ['color_code'])
    ])

X_train_scaled = preprocessor.fit_transform(X_train)
X_test_scaled = preprocessor.transform(X_test)

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_train_scaled_df.hist(figsize=(15,12), bins=30)
plt.tight_layout()
plt.show()

"""# Генерация модели

def build_model(input_dim, layers_config, dropout_rates=None, use_batchnorm=False,
optimizer='adam'):
    model = keras.Sequential()

    for i, units in enumerate(layers_config):
        if i == 0:
            model.add(layers.Dense(units, activation='relu', input_shape=(input_dim,)))
        else:
            model.add(layers.Dense(units, activation='relu'))

        if use_batchnorm:
            model.add(layers.BatchNormalization())

        if dropout_rates and i < len(dropout_rates):
            if dropout_rates[i] > 0:
                model.add(layers.Dropout(dropout_rates[i]))

    model.add(layers.Dense(1))

    model.compile(

```

```

        optimizer=optimizer,
        loss='mse',
        metrics=['mae']
    )

    return model

"""# Обучение и оценка модели"""

class BestModelTracker(keras.callbacks.Callback):
    def __init__(self):
        super().__init__()
        self.best_val_mae = float('inf')

    def on_epoch_end(self, epoch, logs=None):
        val_mae = logs.get('val_mae')

        if val_mae < self.best_val_mae:
            self.best_val_mae = val_mae
            print(f"Epoch {epoch} - New best MAE: {val_mae:.4f}")

def run_experiment(X_train, y_train, layers_config,
                  dropout_rates, use_batchnorm, optimizer,
                  batch_size, epochs=250):

    model = build_model(
        input_dim=X_train.shape[1],
        layers_config=layers_config,
        dropout_rates=dropout_rates,
        use_batchnorm=use_batchnorm,
        optimizer=optimizer
    )

    early_stop = keras.callbacks.EarlyStopping(
        patience=10,
        restore_best_weights=True
    )
    bmt = BestModelTracker()

    history = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=epochs,
        batch_size=batch_size,
        verbose=0,
        callbacks=[early_stop, bmt]
    )

    best_val_mae = min(history.history['val_mae'])

    return best_val_mae, history

```

```
""""# Исследование""""
```

```
def grid_search(X_train, y_train):
    architectures = [
        [32, 16],
        [64, 32],
        [128, 64, 32]
    ]
    batchnorm_options = [True, False]
    dropouts = [
        None,
        [0.2, 0.2],
        [0.3, 0.2, 0.1]
    ]
    optimizers = [
        ("Adam", lambda: keras.optimizers.Adam(0.001)),
        ("RMSprop", lambda: keras.optimizers.RMSprop(0.001)),
        ("SGD", lambda: keras.optimizers.SGD(0.01, momentum=0.9))
    ]

    batch_sizes = [16, 32, 64]
    i=0
    results = []
    for arch, drop, (opt_name, opt_fn), batch, bn in itertools.product(architectures, dropouts,
optimizers, batch_sizes, batchnorm_options):
        if drop is not None and len(drop) != len(arch):
            continue
        i += 1
        print(f"Test {i}: {arch}, drop={drop}, BN={bn}, "
              f"{opt_name}, batch={batch}")
        try:
            val_mae, history = run_experiment(
                X_train, y_train,
                layers_config=arch,
                dropout_rates=drop,
                optimizer=opt_fn(),
                batch_size=batch,
                use_batchnorm=bn,
            )
            model_info = {
                "architecture": arch,
                "dropout": drop,
                "batchnorm": bn,
                "optimizer": opt_name,
                "batch_size": batch,
                "val_mae": val_mae,
                "history": history
            }
            results.append(model_info)

        print(f"Done: {arch}, drop={drop}, BN={bn}, "
              f"{opt_name}, batch={batch} -> MAE={val_mae:.4f}")
```

```

except Exception as e:
    print("Error:", e)

return pd.DataFrame(results)

results_df = grid_search(X_train_scaled, y_train)

print("Общая статистика")
print(results_df['val_mae'].describe())
print(f"Лучшее MAE: {results_df['val_mae'].min():.4f}")
best_row = results_df.loc[results_df['val_mae'].idxmin()]
print("Лучшие параметры:")
print(best_row[['architecture', 'dropout', 'batchnorm', 'optimizer', 'batch_size', 'val_mae']])

df_copy = results_df.copy()

# Преобразуем списки в строки для группировки
df_copy['arch_str'] = df_copy['architecture'].apply(str)

# Группируем по архитектуре и выбираем top 10 по val_mae (наименьшие значения)
top10_per_arch = (df_copy.groupby('arch_str', group_keys=False)
                  .apply(lambda x: x.nsmallest(10, 'val_mae'))
                  .reset_index(drop=True))

print("Топ 10 моделей по архитектуре (лучшие MAE):")
print(top10_per_arch[['architecture', 'dropout', 'batchnorm', 'optimizer', 'batch_size',
'val_mae']].to_string())

plt.figure(figsize=(12, 6))
sns.boxplot(data=top10_per_arch, x='arch_str', y='val_mae',
order=sorted(top10_per_arch['arch_str'].unique()))
plt.title('Распределение лучших MAE по архитектурам (top-10 в каждой)')
plt.xlabel('Архитектура')
plt.ylabel('MAE')
plt.xticks(rotation=45)
plt.show()

def count_params(arch):
    n_in = 11
    total = 0
    prev = n_in
    for units in arch:
        total += prev * units + units
        prev = units
    total += prev * 1 + 1 # выходной слой
    return total

# список уникальных архитектур с их лучшими MAE
arch_mae = {
    str([128,64,32]): 0.516349,
    str([64,32]): 0.531867,

```

```

    str([32,16]):    0.536137
}

arch_params = {str(arch): count_params(arch) for arch in [[128,64,32], [64,32], [32,16]]}

# график сложность vs качество
fig, ax = plt.subplots(figsize=(8, 6))
for arch_str in arch_mae:
    ax.scatter(arch_params[arch_str], arch_mae[arch_str], s=100, label=arch_str)

ax.set_xlabel('Number of parameters')
ax.set_ylabel('Best validation MAE')
ax.set_title('Model complexity vs quality')
ax.legend()
ax.grid(True)
plt.show()

df_plot = results_df.copy()
df_plot['arch_str'] = df_plot['architecture'].apply(str)
df_plot['dropout_str'] = df_plot['dropout'].apply(lambda x: str(x) if x is not None else 'None')

plt.figure(figsize=(15, 10))

# Влияние оптимизатора
plt.subplot(2, 2, 1)
sns.boxplot(data=df_plot, x='optimizer', y='val_mae')
plt.title('MAE по оптимизатору')
plt.xticks(rotation=45)

# Влияние batch size
plt.subplot(2, 2, 2)
sns.boxplot(data=df_plot, x='batch_size', y='val_mae')
plt.title('MAE по batch size')

# Влияние batch normalization
plt.subplot(2, 2, 3)
sns.boxplot(data=df_plot, x='batchnorm', y='val_mae')
plt.title('MAE с/без Batch Normalization')

# Влияние dropout (есть/нет)
plt.subplot(2, 2, 4)
df_plot['has_dropout'] = df_plot['dropout'].apply(lambda x: 'Yes' if x is not None else 'No')
sns.boxplot(data=df_plot, x='has_dropout', y='val_mae')
plt.title('MAE с/без Dropout')

plt.tight_layout()
plt.show()

def train_and_evaluate_model(X_train, y_train, X_test, y_test, layers_config,
                             dropout_rates, use_batchnorm, optimizer,
                             batch_size, epochs=250, verbose=0):
    model = build_model(

```

```

        input_dim=X_train.shape[1],
        layers_config=layers_config,
        dropout_rates=dropout_rates,
        use_batchnorm=use_batchnorm,
        optimizer=optimizer
    )

    early_stop = keras.callbacks.EarlyStopping(
        patience=10,
        restore_best_weights=True,
    )
    bmt = BestModelTracker()

    history = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=epochs,
        batch_size=batch_size,
        verbose=verbose,
        callbacks=[early_stop, bmt]
    )

    # Предсказание на тесте
    y_pred = model.predict(X_test, verbose=0).flatten()
    test_mae = mean_absolute_error(y_test, y_pred)
    test_mse = mean_squared_error(y_test, y_pred)
    test_r2 = r2_score(y_test, y_pred)

    return history, test_mae, test_mse, test_r2

def run_multiple_trials(X_train, y_train, X_test, y_test, layers_config,
                        dropout_rates, use_batchnorm, optimizer,
                        batch_size, n_trials=10, epochs=250):
    results = []
    for i in range(n_trials):
        print(f"Trial {i+1}/{n_trials}")
        _, test_mae, test_mse, test_r2 = train_and_evaluate_model(
            X_train, y_train, X_test, y_test,
            layers_config, dropout_rates, use_batchnorm, optimizer(),
            batch_size, epochs, verbose=0
        )
        results.append({
            'trial': i+1,
            'test_mae': test_mae,
            'test_mse': test_mse,
            'test_r2': test_r2
        })
    return pd.DataFrame(results)

optimal_config = {
    'layers_config': [64, 32],
    'dropout_rates': None,

```

```

        'use_batchnorm': False,
        'optimizer': lambda: keras.optimizers.SGD(0.01, momentum=0.9),
        'batch_size': 32
    }

    trials_df = run_multiple_trials(X_train_scaled, y_train, X_test_scaled, y_test,
                                    **optimal_config, n_trials=10, epochs=250)

    print(trials_df[['test_mae', 'test_mse', 'test_r2']].describe())

    # График разброса тестовых MAE
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(trials_df['trial'], trials_df['test_mae'], 'o-')
    plt.xlabel('Trial')
    plt.ylabel('Test MAE')
    plt.title('Stability of test MAE over 10 runs')
    plt.axhline(y=trials_df['test_mae'].mean(), color='r', linestyle='--', label=f'Mean: {trials_df["test_mae"].mean():.4f}')
    plt.legend()

    plt.subplot(1, 2, 2)
    sns.boxplot(y=trials_df['test_mae'])
    plt.ylabel('Test MAE')
    plt.title('Distribution of test MAE')
    plt.tight_layout()
    plt.show()

    history, test_mae, test_mse, test_r2 = train_and_evaluate_model(
        X_train_scaled, y_train, X_test_scaled, y_test,
        layers_config=[64, 32],
        dropout_rates= None,
        use_batchnorm=False,
        optimizer=keras.optimizers.SGD(0.01, momentum=0.9),
        batch_size=32,
        epochs=250,
        verbose=0
    )

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    ax1.plot(history.history['loss'], label='Train loss (MSE)')
    ax1.plot(history.history['val_loss'], label='Validation loss (MSE)')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('MSE')
    ax1.legend()
    ax1.set_title('Loss curves (optimal model)')

    ax2.plot(history.history['mae'], label='Train MAE')
    ax2.plot(history.history['val_mae'], label='Validation MAE')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('MAE')

```

```
ax2.legend()  
ax2.set_title('MAE curves (optimal model)')  
  
plt.tight_layout()  
plt.show()
```